

小结

26

本章介绍了足够多的 C++ 语言的知识，以使你能够编译、运行简单的 C++ 程序。我们看到了如何定义一个 main 函数，它是操作系统执行你的程序的调用入口。我们还看到了如何定义变量，如何进行输入输出，以及如何编写 if、for 和 while 语句。本章最后介绍了 C++ 中最基本的特性——类。在本章中，我们看到了，对于其他人定义的一个类，我们应该如何创建、使用其对象。在后续章节中，我们将介绍如何定义自己的类。

术语表

参数 (实参, argument) 向函数传递的值。

赋值 (assignment) 抹去一个对象的当前值，用一个新值取代之。

程序块 (block) 零条或多条语句的序列，用花括号包围。

缓冲区 (buffer) 一个存储区域，用于保存数据。IO 设施通常将输入（或输出）数据保存在一个缓冲区中，读写缓冲区的动作与程序中的动作是无关的。我们可以显式地刷新输出缓冲，以便强制将缓冲区中的数据写入输出设备。默认情况下，读 cin 会刷新 cout；程序非正常终止时也会刷新 cout。

内置类型 (built-in type) 由语言定义的类型，如 int。

Cerr 一个 ostream 对象，关联到标准错误，通常写入到与标准输出相同的设备。默认情况下，写到 cerr 的数据是不缓冲的。cerr 通常用于输出错误信息或其他不属于程序正常逻辑的输出内容。

字符串字面值常量 (character string literal) 术语 string literal 的另一种叫法。

cin 一个 istream 对象，用来从标准输入读取数据。

类 (class) 一种用于定义自己的数据结构及其相关操作的机制。类是 C++ 中最基本的特性之一。标准库类型中，如 istream 和 ostream 都是类。

类类型 (class type) 类定义的类型。类名即为类型名。

clog 一个 ostream 对象，关联到标准错误。默认情况下，写到 clog 的数据是被缓冲的。clog 通常用于报告程序的执行信息，存入一个日志文件中。

注释 (comment) 被编译器忽略的程序文本。C++ 有两种类型的注释：单行注释和界定符对注释。单行注释以 // 开始，从 // 到行尾的所有内容都是注释。界定符对注释以 /* 开始，其后的所有内容都是注释，直至遇到 */ 为止。

条件 (condition) 求值结果为真或假的表达式。通常用值 0 表示假，用非零值表示真。

cout 一个 ostream 对象，用于将数据写入标准输出。通常用于程序的正常输出内容。

花括号 (curly brace) 花括号用于划定程序块边界。左花括号 ({) 为程序块开始，右花括号 (}) 为结束。

数据结构 (data structure) 数据及其上所允许的操作的一种逻辑组合。

编辑-编译-调试 (edit-compile-debug) 使程序能正确执行的开发过程。

文件结束符 (end-of-file) 系统特定的标识，指出文件中无更多数据了。

表达式 (expression) 最小的计算单元。一个表达式包含一个或多个运算对象，通常还包含一个或多个运算符。表达式求值会产生一个结果。例如，假设 i 和 j 是 int 对象，则 i+j 是一个表达式，它产生两个

27

int 值的和。

for 语句 (for statement) 迭代语句，提供重复执行能力。通常用来将一个计算反复执行指定次数。

函数 (function) 具名的计算单元。

函数体 (function body) 语句块，定义了函数所执行的动作。

函数名 (function name) 函数为人所知的名字，也用来进行函数调用。

头文件 (header) 使类或其他名字的定义可被多个程序使用的一种机制。程序通过 `#include` 指令使用头文件。

if 语句 (if statement) 根据一个特定条件的值进行条件执行的语句。如果条件为真，执行 if 语句体。否则，执行 else 语句体（如果存在的话）。

初始化 (initialize) 在一个对象创建的时候赋予它一个值。

iostream 头文件，提供了面向流的输入输出的标准库类型。

istream 提供了面向流的输入的库类型。

库类型 (library type) 标准库定义的类型， 接一个分号就是一条语句；其他类型的语句包括语句块、if 语句、for 语句和 while 语句，所有这些语句内都包含其他语句。

main 操作系统执行一个 C++ 程序时所调用的函数。每个程序必须有且只有一个命名为 main 的函数。

操纵符 (manipulator) 对象，如 `std::endl`，在读写流的时候用来“操纵”流本身。

成员函数 (member function) 类定义的操作。通常通过调用成员函数来操作特定对象。

方法 (method) 成员函数的同义术语。

命名空间 (namespace) 将库定义的名字放在一个单一位置的机制。命名空间可以帮助避免不经意的名字冲突。C++ 标准库定义的名字在命名空间 `std` 中。

ostream 标准库类型，提供面向流的输出。

形参列表 (parameter list) 函数定义的一部分，指出调用函数时可以使用什么样的实参，可能为空列表。

返回类型 (return type) 函数返回值的类型。

源文件 (source file) 包含 C++ 程序的文件。

标准错误 (standard error) 输出流，用于报告错误。标准输出和标准错误通常关联到程序执行所在的窗口。

标准输入 (standard input) 输入流，通常与程序执行所在窗口相关联。

标准库 (standard library) 一个类型和函数的集合，每个 C++ 编译器都必须支持。标准库提供了支持 IO 操作的类型。C++ 程序员倾向于用“库”指代整个标准库，还倾向于用库类型表示标准库的特定部分，例如用“`iostream` 库”表示标准库中定义 IO 类的部分。

标准输出 (standard output) 输出流，通常与程序执行所在窗口相关联。

语句 (statement) 程序的一部分，指定了当程序执行时进行什么动作。一个表达式接一个分号就是一条语句；其他类型的语句包括语句块、if 语句、for 语句和 while 语句，所有这些语句内都包含其他语句。

std 标准库所使用的命名空间。`std::cout` 表示我们要使用定义在命名空间 `std` 中的名字 `cout`。

字符串常量 (string literal) 零或多个字符组成的序列，用双引号包围 ("a string literal")。

未初始化的变量 (uninitialized variable) 未赋予初值的变量。类类型的变量如果没有指定初值，则按类定义指定的方式进行初始化。定义在函数内部的内置类型变量默认是不初始化的，除非有显式的初始化语句。试图使用一个未初始化变量的值是错误的。未初始化变量是 bug 的常见成因。

变量 (variable) 具名对象。

while 语句 (while statement) 迭代语句，提供重复执行直至一个特定条件为假的机制。循环体会执行零次或多次，依赖于循环条件求值结果。

()运算符 (operator) 调用运算符。跟随在函数名之后的一对括号“()”，起到调用函数的效果。传递给函数的实参放置在括号内。

++运算符 (++ operator) 递增运算符。将运算对象的值加 1，`++i` 等价于 `i=i+1`。

+=运算符 (+= operator) 复合赋值运算符，将右侧运算对象加到左侧运算对象上；`a+=b` 等价于 `a=a+b`。

.运算符 (. operator) 点运算符。左侧运算对象必须是一个类类型对象，右侧运算对象必须是此对象的一个成员的名字。运算结果即为该对象的这个成员。

::运算符 (:: operator) 作用域运算符。其用处之一是访问命名空间中的名字。例如，`std::cout` 表示命名空间 `std` 中的名字 `cout`。

=运算符 (= operator) 将右侧运算对象的值赋予左侧运算对象所表示的对象。

--运算符 (-- operator) 递减运算符。将运算对象的值减 1，`--i` 等价于 `i=i-1`。

<<运算符 (<< operator) 输出运算符。将

右侧运算对象的值写到左侧运算对象表示的输出流：`cout << "hi"` 表示将 `hi` 写到标准输出。输出运算符可以连接：`cout << "hi" << "bye"` 表示将输出 `hibye`。

>>运算符 (>> operator) 输入运算符。从左侧运算对象所指定的输入流读取数据，存入右侧运算对象中：`cin >> i` 表示从标准输入读取下一个值，存入 `i` 中。输入运算符可以连接：`cin >> i >> j` 表示先读取一个值存入 `i`，再读取一个值存入 `j`。

#include 头文件包含指令，使头文件中代码可被程序使用。

==运算符 (== operator) 相等运算符。检测左侧运算对象是否等于右侧运算对象。

!=运算符 (!= operator) 不等运算符。检测左侧运算对象是否不等于右侧运算对象。

<=运算符 (<= operator) 小于等于运算符。检测左侧运算对象是否小于等于右侧运算对象。

<运算符 (< operator) 小于运算符。检测左侧运算对象是否小于右侧运算对象。

>=运算符 (>= operator) 大于等于运算符。检测左侧运算对象是否大于等于右侧运算对象。

>运算符 (> operator) 大于运算符。检测左侧运算对象是否大于右侧运算对象。

第 I 部分

C++ 基础

内容

第 2 章 变量和基本类型	29
第 3 章 字符串、向量和数组	73
第 4 章 表达式	119
第 5 章 语句	153
第 6 章 函数	181
第 7 章 类	227

任何常用的编程语言都具备一组公共的语法特征，不同语言仅在特征的细节上有所区别。要想学习并掌握一种编程语言，理解其语法特征的实现细节是第一步。最基本的特征包括：

- 整型、字符型等内置类型
- 变量，用来为对象命名
- 表达式和语句，用于操纵上述数据类型的具体值
- if 或 while 等控制结构，这些结构允许我们有选择地执行一些语句或者重复地执行一些语句
- 函数，用于定义可供随时调用的计算单元

大多数编程语言通过两种方式来进行进一步补充其基本特征：一是赋予程序员自定义数据类型的权利，从而实现对语言的扩展；二是将一些有用的功能封装成库函数提供给程序员。

30

与大多数编程语言一样，C++的对象类型决定了能对该对象进行的操作，一条表达式是否合法依赖于其中参与运算的对象的类型。一些语言，如 Smalltalk 和 Python 等，在程序运行时检查数据类型；与之相反，C++是一种静态数据类型语言，它的类型检查发生在编译时。因此，编译器必须知道程序中每一个变量对应的数据类型。

C++提供了一组内置数据类型、相应的运算符以及为数不多的几种程序流控制语句，这些元素共同构成了 C++语言的基本形态。以这些元素为基础，我们可以编写出规模庞大、结构复杂、用于解决实际问题的软件系统。仅就 C++的基本形态来说，它是一种简单的编程语言，其强大的能力显示于它对程序员自定义数据结构的支持。这种支持作用巨大，显而易见的一个事实是，C++语言的缔造者无须洞悉所有程序员的要求，而程序员恰好可以通过自定义新的数据结构来使语言满足他们各自的需求。

C++中最重要的语法特征应该就是类了，通过它，程序员可以定义自己的数据类型。为了与 C++的内置类型区别开来，它们通常被称为“类类型 (class type)”。在一些编程语言中，程序员自定义的新类型仅能包含数据成员；另外一些语言，比如 C++，则允许新类型中既包含数据成员，也包含函数成员。C++语言主要的一个设计目标就是让程序员自定义的数据类型像内置类型一样好用。基于此，标准 C++库实现了丰富的类和函数。

本书第 I 部分的主题是学习 C++语言的基础知识，这也是掌握 C++语言的第一步。第 2 章详述内置类型，并初步介绍了自定义数据类型的方法。第 3 章介绍了两种最基本的数据类型：字符串和向量。C++和许多编程语言所共有的一种底层数据结构——数组也在本章有所提及。接下来，第 4~6 章依次介绍了表达式、语句和函数。作为第 I 部分的最后一章，第 7 章描述了如何构建我们自己的类，完成这一任务需要综合运用之前各章所介绍的知识。

第 2 章

变量和基本类型

内容

2.1 基本内置类型	30
2.2 变量	38
2.3 复合类型	45
2.4 const 限定符	53
2.5 处理类型	60
2.6 自定义数据结构	64
小结	69
术语表	69

数据类型是程序的基础：它告诉我们数据的意义以及我们能在数据上执行的操作。

C++语言支持广泛的数据类型。它定义了几种基本内置类型（如字符、整型、浮点数等），同时也为程序员提供了自定义数据类型的机制。基于此，C++标准库定义了一些更加复杂的数据类型，比如可变长字符串和向量等。本章将主要讲述内置类型，并带领大家初步了解 C++语言是如何支持更复杂数据类型的。